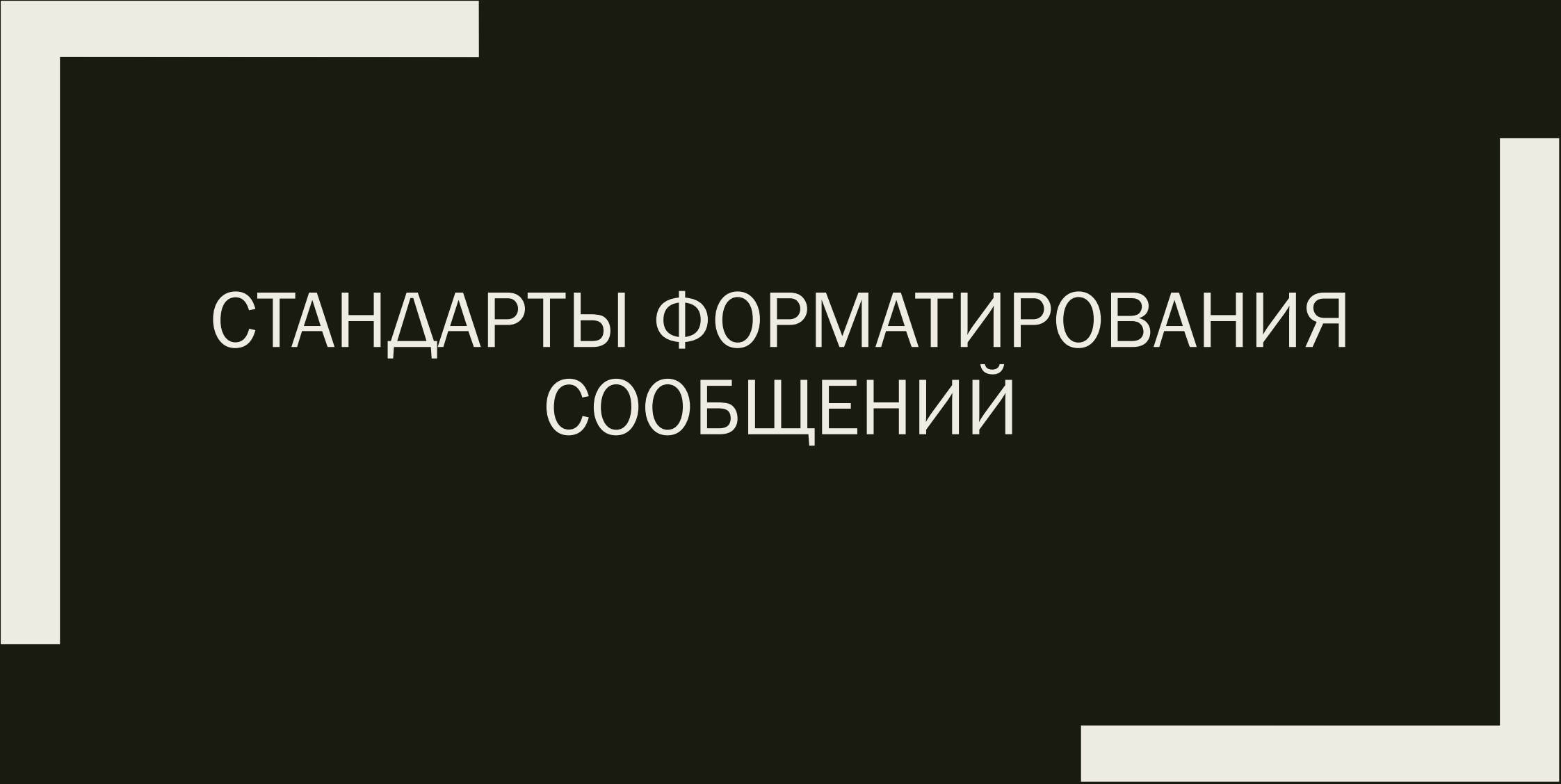




СТАНДАРТЫ ФОРМАТИРОВАНИЯ СООБЩЕНИЙ

Стандарты форматирования сообщений

В современном мире программирования и веб-разработки **ключом к успешному общению между различными приложениями и системами является надежная и быстрая передача информации.** С каждым днем разработчикам приходится все чаще иметь дело не только с простыми конфигурационными параметрами, но и с передачей сложных данных для общения с внешними API и веб-сервисами. Здесь **на передний план выходят два основных формата обмена данными: XML (eXtensible Markup Language) и JSON (JavaScript Object Notation).** Они занимают центральное место в этой области, предоставляя разработчикам надежные средства **для передачи, хранения и структурирования информации.**

Стандарты форматирования сообщений

В этой сфере форматы XML и JSON, каждый со своей уникальной структурой и подходом, предоставляют средства для организации данных таким образом, чтобы они могли быть легко восприняты как человеком, так и машиной.

Что такое XML

XML (eXtensible Markup Language), что означает «расширяемый язык разметки», представляет собой универсальный язык разметки, который был разработан для обмена структурированными данными между системами. Будучи представленным в 1998 году, XML установил новые стандарты хранения и передачи информации.

Что такое XML

XML применяется для организации информации в структуру, описываемую через иерархии с помощью тегов. Теги служат в качестве определителей элементов данных и устанавливают между ними связи. Каждый тег имеет свое название и может содержать атрибуты и вложенные элементы:

```
<название_элемента атрибут="значение">
```

Содержимое элемента

```
</название_элемента>
```

Что такое XML

В этом примере `<название_элемента>` и `</название_элемента>` — это открывающий и закрывающий теги, которые определяют начало и конец элемента, соответственно.

Что такое XML

Основными компонентами XML-документа являются элементы, атрибуты и текстовое содержимое:

- Элементы представляют основные блоки данных в XML. Они ограничиваются тегами, например:

```
<книга>  
  <название>XML для начинающих</название>  
  <автор>Иван Петров</автор>  
</книга>
```

Что такое XML

- Атрибуты представляют дополнительную информацию об элементе. Они указываются внутри тега и имеют вид имя="значение":

```
<книга категория="Программирование">  
  <название>XML для начинающих</название>  
  <автор>Иван Петров</автор>  
</книга>
```


Что такое XML

- Элементы также могут содержать текст:

<книга>

 <название>XML для начинающих</название>

 <автор>Иван Петров</автор>

 <описание>Эта книга предназначена для тех, кто только начинает изучать XML</описание>

</книга>

Что такое XML

Одним из ключевых достоинств XML является его возможность поддерживать схемы, что обеспечивает способность определения строгой структуры данных и их последующую валидацию. XML-схема предоставляет набор правил и ограничений, которым должен соответствовать XML-документ для того, чтобы быть признанным валидным. Она указывает допустимые элементы, атрибуты, последовательность и уровень вложенности элементов, а также типы данных для текстового контента. Для описания схем в XML используется два основных подхода — это DTD (Document Type Definition) и XSD (XML Schema Definition).

Что такое XML

DTD, или документ типа определения — это структура, оговоренная в самом XML-документе или размещенная отдельно, которая содержит правила для форматирования и проверки корректности данных. DTD позволяет указать разрешенные элементы, их взаимоотношения, атрибуты и их типы данных.

Что такое XML

XSD, известное как XML Schema Definition, предлагает более продвинутый и универсальный подход к организации структуры данных в формате XML по сравнению с DTD. Основным преимуществом XSD является его способность выполнять более тонкую и объемную проверку данных в XML-документе. XSD представлен отдельным документом, написанным на языке XML, и поддерживает большое количество типов данных.

Применение XML

XML широко применяется в различных областях, предоставляя гибкий способ организации и передачи структурированных данных:

Web-сервисы: XML является стандартным форматом для обмена информацией между веб-сервисами. Это обусловлено тем, что XML поддерживает сложные структуры и может быть использован для представления любых данных, которые можно описать в виде древовидной структуры.

Применение XML

Например, SOAP (Simple Object Access Protocol) — это стандартный протокол обмена сообщениями на основе XML. Web-сервисы используют SOAP для отправки в формате XML через сеть.

Другой пример, Amazon Web Services: большинство AWS-сервисов предоставляют API на основе XML. Это включает в себя S3 (для хранения и извлечения объектов), EC2 (для управления виртуальными серверами), SimpleDB (для работы с базами данных) и многие другие.

Применение XML

Синдикация контента: данный термин означает распространение информации с одного сайта на большее количество ресурсов.

RSS (Really Simple Syndication) и Atom — это популярные XML-форматы для новостных лент. Они представляют собой простой и стандартизированный способ предоставления пользователю возможности подписаться на обновления сайта: новые статьи, блоги, новости и т.п.

Форматы RSS и Atom позволяют веб-сайтам своего рода «упаковывать» свои обновления в стандартный XML-формат, который затем могут читать различные клиентские приложения — так называемые агрегаторы новостей.

Применение XML

- **Форматы файлов:** некоторые программы используют XML для представления своих данных.

Например, программы Microsoft Office (Word, Excel) используют формат Office Open XML для своих файлов. Также SVG (Scalable Vector Graphics) — язык для описания двумерной векторной графики в формате XML.

Применение XML

Конфигурационные файлы: XML часто используется для хранения конфигурационных данных программ и сервисов, так как он удобно структурирован и легко читается как человеком, так и машиной.

Например, Hibernate конфигурационный файл (`hibernate.cfg.xml`). Hibernate — это фреймворк для объектно-реляционного отображения (ORM) на Java. Конфигурационный файл XML используется для определения свойств соединения с базой данных, маппинга классов и других настроек.

Другой пример, Spring Bean Configuration файл: в Spring Framework конфигурационный файл XML служит для определения beans (объектов), которые управляются Spring IoC контейнером.

Применение XML

Базы данных: хотя XML может быть не самым оптимальным способом хранения данных из-за своей избыточности, он все же используется в базах данных для обмена информацией и иногда даже для хранения данных.

Например, Oracle XML DB — это функциональность базы данных Oracle, которая позволяет хранить XML-документы, как реляционные столбцы. Она также поддерживает XQuery для выполнения запросов к XML-данным.

Другой пример, PostgreSQL — эта система базы данных предоставляет поддержку XML, позволяя хранить XML-документы и выполнять запросы с использованием XPath.

Применение XML

Сетевые протоколы: XML является частью множества протоколов, так как он предоставляет удобный, универсальный и масштабируемый способ представления данных.

Например, XMPP (Extensible Messaging and Presence Protocol) — сетевой протокол, который использует XML для обмена мгновенными сообщениями и информацией о присутствии (например, онлайн или офлайн статус пользователя) в реал-тайме.

Применение XML

Электронная бизнес-документация: XML играет важную роль в электронной бизнес-документации, как надежный и универсальный формат для представления структурированных данных.

Например, UBL (Universal Business Language) — это набор стандартных XML-форматов для обмена типичными бизнес-документами, такими как счета заказы, приемы и отправки товаров.

Что такое JSON

JSON, который также известен как JavaScript Object Notation, представляет собой компактный и простой в использовании формат передачи данных. Этот формат был впервые представлен в 2001 году. Он базируется на синтаксисе объекта в JavaScript, но при этом не ограничивается использованием только в этом языке программирования.

Что такое JSON

Два основных типа структур данных в JSON — это объекты и массивы:

В JSON объекты являются беспорядочным набором сочетаний «ключ-значение». Между ключами (названиями) и значениями ставятся двоеточия, а пары разделяются запятой. Открывающая и закрывающая фигурные скобки обозначают начало и конец объекта:

```
{  
  "название": "JSON для начинающих",  
  "автор": "Иван Петров"  
}
```

Что такое JSON

Массивы представляют собой уже упорядоченный список значений. Значения в массиве разделяются запятой. Массивы начинаются с открывающей квадратной скобки и заканчиваются закрывающей квадратной скобкой:

```
[  
  "яблоко",  
  "апельсин",  
  "банан"  
]
```

Что такое JSON

Эти две структуры могут комбинироваться для создания более сложных иерархий данных в JSON. Например, объекты могут содержать вложенные объекты или массивы, а массивы могут содержать объекты или другие массивы.

Одной из ключевых характеристик JSON является его способность представлять разнообразные типы данных, используя ограниченный набор примитивных базовых типов, таких как строки, числа, логические значения, массивы и вложенные объекты. Эта гибкость делает его идеальным для передачи структурированных данных в приложениях, веб-сервисах и API.

Что такое JSON

Кроме того, для более формального описания структуры данных в формате JSON существует спецификация JSON Schema, которая позволяет определить правила и требования к данным в формате JSON, обеспечивая их валидацию:

```
{
  "structure": "object",
  "parameters": {
    "fullname": { "structure": "unicode" },
    "years": { "structure": "numeric" }
  },
  "compulsory": ["fullname", "years"]
}
```

Применение JSON

Изучим некоторые практические случаи использования JSON:

Веб-API: большинство веб-сервисов используют JSON для отправки данных в своих API. Например, при использовании RESTful API, сервер часто отправляет данные в формате JSON.

Применение JSON

В качестве примера, YouTube Data API, если вы хотите получить информацию о видео, вы можете отправить запрос GET, указав идентификатор видео. В ответ вы получите JSON-объект со всеми данными о видео.

Еще один пример, GitHub API, который позволяет разработчикам взаимодействовать с платформой GitHub, включая получение информации о репозиториях, пользователях, коммитах и других аспектах службы. Данные возвращаются в формате JSON.

Применение JSON

Базы данных: JSON также поддерживается многими типами баз данных. MongoDB, одна из наиболее распространенных NoSQL-баз данных, для хранения документов использует формат BSON, двоичная форма JSON.

Некоторые классические SQL БД, такие как PostgreSQL и MySQL, также предлагают функции для работы с JSON. Они могут хранить JSON в качестве строки и предоставляют встроенные функции для работы с данными.

Применение JSON

Конфигурационные файлы: JSON часто используется для сохранения конфигураций в веб-приложениях. В современной web-разработке файл `package.json` служит для хранения информации о проекте и его зависимостях в Node.js проектах.

Другой пример, Webpack, который является современным сборщиком модулей JavaScript и использует файл `webpack.config.js` для своих конфигурационных настроек. Хотя сам файл является файлом JavaScript, он часто содержит JSON-подобные объекты для конфигурации.

Применение JSON

Передача данных между клиентом и сервером: при отправке запросов на сервер (например, отправки формы или запроса данных), сервер обычно отвечает в формате JSON. Эти данные затем могут быть легко обработаны и отображены на стороне пользователя.

Веб-браузеры и серверные языки программирования, такие как Node.js, имеют встроенные функции для преобразования данных JSON в объекты JavaScript и обратно.

Применение JSON

Настройка визуализации данных: JSON идеально подходит для структурирования данных для отображения на графиках или диаграммах.

Многие библиотеки для визуализации, такие как D3.js, Plotly или Highcharts, могут принимать данные в формате JSON для создания графиков и диаграмм. Метод структурирования данных будет варьироваться в зависимости от того, какая библиотека используется, так как каждая библиотека имеет свой уникальный подход к работе с данными.

Применение JSON

Использование данных в JavaScript: JSON, как следует из названия, основан на объектах JavaScript. Это упрощает работу с данными, поскольку они могут быть преобразованы из JSON в JavaScript и обратно без необходимости разбора или форматирования данных.

Данные для тестирования: JSON также может быть использован для создания фейковых данных для тестирования веб-приложений.

Преимущества XML

Поддержка пространства имен: пространства имен в XML используются для избегания конфликтов при определении тегов или элементов, которые могут совпадать по имени в XML документе. Это особенно значимо в больших системах или при интеграции сразу нескольких источников данных, где вероятность совпадения имен велика.

Расширяемость: XML может быть расширен для создания своих собственных тегов и структуры документов, что делает его гибким решением для различных задач.

Поддержка стилей: XML поддерживает XSL (eXtensible Stylesheet Language), что позволяет преобразовывать и стилизовать XML-документы для отображения в различных форматах, таких как HTML, PDF и других.

Метаданные и аннотации: XML позволяет включать метаданные прямо в документ, что может быть важно для некоторых приложений. В JSON для этого необходимо создавать дополнительные структуры.

Безопасность: XML поддерживает цифровые подписи и шифрование, что делает его более безопасным для обмена конфиденциальными данными.

Преимущества JSON

Понятность и доступность: JSON обеспечивает более упрощенную и понятную структуру в сравнении с XML, что делает его более комфортным для разработчиков в процессе чтения и написания кода.

Производительность: JSON обычно требует меньше места по сравнению с XML, благодаря отсутствию закрывающих тегов. Это способствует более быстрой и оптимальной передаче данных.

Поддержка структур данных: JSON поддерживает базовые типы данных, такие как числа, строки и булевы значения, а также комплексные типы, включая массивы и объекты. Для отображения сложных структур данных в XML требуется больше ресурсов и кодирования.

Поддержка многочисленных языков программирования: например, веб-приложение, созданное на JavaScript (клиентская сторона), и серверная сторона, реализованная на Python. Данные могут быть легко форматированы в JSON на клиентской стороне и переданы на сервер для обработки. Такая совместимость формата данных с различными языками обеспечивает гибкость и удобство при разработке приложений, использующих JSON.

Ограничения XML

Сложность: XML вводит множество концепций, таких как DTD (определения типов документов), пространства имен, XSLT (язык преобразований XML) и т.д., которые могут быть сложными для понимания и обучения, особенно по сравнению с более простым JSON.

Необходимость парсинга: XML-данные должны быть проанализированы (или «приведены в порядок») перед использованием, что может повлечь издержки на CPU и время обработки.

Более сложная структура данных: в отличие от JSON, который может прямо отображать сложные структуры данных, в XML требуется больше усилий для кодирования таких структур.

Отсутствие поддержки «родных» типов данных: XML не рассматривает данные как определенные типы (например, числа, булевы значения), все они интерпретируются как текст.

Ограничения JSON

Отсутствие комментариев: несмотря на преимущества JSON по сравнению с XML, стандартный синтаксис JSON не предусматривает возможности добавления комментариев. Это может оказаться недостатком при необходимости включения пояснений или аннотаций к коду.

Ограниченная поддержка для типов данных: JSON поддерживает ограниченное количество типов данных. Например, нет поддержки дат и времени, что может привести к несоответствиям между языками программирования при обмене данными.

Отсутствие схемы: в отличие от XML, JSON не имеет встроенной поддержки определения и валидации структуры данных. Это может усложнить установление и обеспечение правил.

Сравнение JSON и XML

Характеристика	JSON	XML
Синтаксис	Простой и лаконичный, легко читаемый	Более формальный, много символов, сложный для восприятия без определенного опыта
Поддержка комментариев	В основном отсутствуют	Поддерживается с использованием <code><!-- комментарий --></code>
Производительность	Часто более быстрый при обработке в браузере	Может быть менее эффективным при больших объемах данных
Парсинг	Более быстро и легко парсится, что делает его предпочтительным для быстрых веб-приложений и RESTful API	Требует больше ресурсов для парсинга и анализа данных
Поддержка пространства имен	Не поддерживает	Поддерживает, что облегчает работу в больших системах и при интеграции различных источников данных